

Desarrollo de Aplicaciones Multiplataforma



Florida

Universitària

Proyecto Intermodular

Título: TranquiCar

Autores: Ángel, Andrea y Joan

Curso Académico: 2DAM

Declaración responsable de originalidad

Nosotros, los abajo firmantes, alumnos del CFGS en Desarrollo de Aplicaciones Multiplataforma (Florida Centre de Formació, curso 2025/2026), declaramos bajo nuestra responsabilidad que el presente proyecto ha sido realizado íntegramente por nuestro grupo de trabajo.

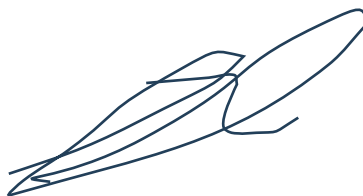
Afirmamos que:

1. El contenido, desarrollo, código fuente y documentación incluidos en esta memoria son fruto de nuestro propio esfuerzo y aprendizaje.
2. No hemos copiado ni reproducido trabajos de terceros sin la debida citación y referencia.
3. En relación con el uso de herramientas de Inteligencia Artificial (IA), manifestamos que:
 - El proyecto y su memoria son originales y elaborados por nosotros.
 - En caso de haber utilizado IA como apoyo (por ejemplo, para obtener ideas, resolver dudas técnicas o generar ejemplos), su contribución ha sido puntual, supervisada y siempre debidamente contrastada, de modo que no sustituye nuestro propio trabajo.
 - Nos responsabilizamos de la autenticidad, validez y corrección del contenido presentado.
4. Somos conscientes de que la falsedad en esta declaración puede conllevar consecuencias académicas, de acuerdo con la normativa del centro y la legislación vigente.

En Catarroja, a 20 de Mayo de 2026

Firmas de los autores:

Ángel Valero Figueroa



Andrea Alfonso Rubio



Joan Escrivá Moliner

Declaración responsable de originalidad	2
Índice	3
Índice de figuras.....	4
1. Resumen del proyecto	5
2. Justificación y objetivos del proyecto	7
3. Desarrollo del proyecto	8
3.1 Análisis del mercado y posible modelo de negocio	8
3.2 Metodologías utilizadas	9
3.3 Descripción de los componentes de la aplicación	10
3.4 Principales problemas y soluciones	13
3.5 Resultados obtenidos	16
4. Conclusiones	21
5. Líneas de trabajo futuro	21
6. Bibliografía	23

Índice de figuras

Figura 1. Pantalla de inicio de sesión

Figura 2. Pantalla de registro

Figura 3. Pantalla de recuperación de contraseña

Figura 4. Pantalla principal del garaje

Figura 5. Vista detallada de vehículo

Figura 6. Historial de mantenimientos

Figura 7. Pantalla de recordatorios

Figura 8. Formulario de añadir mantenimiento

Figura 9. Formulario de añadir recordatorio

Figura 10. Arquitectura general de la aplicación

Figura 11. Diagrama entidad-relación de la base de datos

1. Resumen del proyecto

Español:

El proyecto consiste en una aplicación súper intuitiva para llevar el control del mantenimiento de coches y motos. La idea principal es acabar con los típicos olvidos de cuándo toca revisar el vehículo, ayudando a los usuarios a ahorrar en averías graves y a alargar la vida útil de su motor. Todo centralizado en un "garaje virtual" donde tendremos todos los vehículos a los que queremos hacer un seguimiento, permitiendo añadir tantos como se quiera con sus datos básicos como marca, modelo, año y kilómetros actuales. Para dar forma a la app, el desarrollo se ha centrado en crear herramientas muy fáciles de usar e intuitivas para todo tipo de usuarios. Al entrar en el perfil de un vehículo concreto del garaje, se accede a su información exclusiva, por un lado, está el registro de mantenimientos, como cambios de aceite, filtros, neumáticos o frenos, donde registras la fecha, el coste y los kilómetros. Al ser interactivo sirve tanto para registrar lo que ya se ha hecho como para planificar lo que está por hacer, y puedes editar o eliminar cualquier entrada en cualquier momento. Por otro lado, la lógica de la app incluye un sistema que calcula alertas automáticas, ya sea por tiempo o por kilómetros. Estas alertas van a un historial exclusivo de recordatorios que también se puede modificar o eliminar. Como resultado, se ha conseguido una herramienta muy práctica que descarta al clásico papel y boli. Los usuarios tienen ahora el control total y flexible de su coche o moto, sabiendo exactamente qué se ha tocado, cuánto se han gastado y, lo más importante, recibiendo avisos automáticos para que el vehículo esté siempre a punto.

Inglés:

This project consists of the development of a highly intuitive application designed to manage car and motorcycle maintenance efficiently. The main objective of the app is to prevent users from forgetting important vehicle service intervals, helping them avoid expensive mechanical failures and extend the lifespan of their engines. The application centralizes all information in a “virtual garage,” where users can add and manage multiple vehicles by entering essential details such as brand, model, year, and current mileage. The development process focused on creating a user-friendly and accessible interface suitable for all types of users. Each vehicle profile provides exclusive information and tools to manage maintenance records, including oil changes, filter replacements, tire changes, and brake inspections. Users can register the date, mileage, and cost of each maintenance task, while also planning future services. The interactive system allows users to edit or delete entries at any time, ensuring flexibility and accuracy.

In addition, the application incorporates an automatic alert system that calculates maintenance reminders based on either time intervals or mileage. These notifications are stored in a dedicated reminder history, which can also be modified or deleted by the user. As a result, the project has successfully delivered a practical and modern alternative to traditional paper-based maintenance tracking methods. Users now have complete control over their vehicles, with easy access to maintenance history, expenses, and automatic reminders that help keep their cars and motorcycles in optimal condition at all times.

2. Justificación y objetivos del proyecto

El mantenimiento de vehículos es una tarea fundamental para garantizar la seguridad del conductor y el correcto funcionamiento del automóvil o motocicleta. Sin embargo, durante el análisis previo del proyecto se detectó que las soluciones actuales utilizadas por la mayoría de usuarios particulares se encuentran desactualizadas o resultan poco prácticas. Muchas personas continúan utilizando métodos tradicionales, como guardar facturas en papel o utilizar pegatinas recordatorias, mientras que otras recurren a hojas de cálculo complejas y difíciles de gestionar. Aunque existen aplicaciones destinadas al mantenimiento de vehículos, la mayoría incorporan demasiadas funcionalidades innecesarias para un usuario común, lo que termina generando una experiencia confusa y poco intuitiva.

A partir de esta problemática surge TranquiCar, una aplicación centrada en ofrecer una solución sencilla, clara y eficaz. El principal objetivo del proyecto es crear una aplicación capaz de realizar de forma eficiente las tareas realmente importantes para el usuario, como registrar mantenimientos y generar recordatorios automáticos para evitar olvidos. El proyecto está orientado principalmente a usuarios particulares que desean llevar un control organizado del estado y mantenimiento de sus vehículos de manera rápida y accesible. Nuestros stakeholders del proyecto son Paco Segura Mateu, Nacho Muñoz Rodríguez y Eva Ramón Canet, quienes han participado en la evolución de la aplicación. Entre los objetivos conseguidos destaca la implementación de un sistema de gestión de perfil y garaje virtual, permitiendo a los usuarios registrar múltiples vehículos y visualizar su estado general. Además, se ha desarrollado un sistema CRUD para registrar mantenimientos como cambios de aceite, frenos o neumáticos, almacenando información relevante como fecha, coste y kilometraje. Finalmente, se ha implementado una lógica de recordatorios automáticos que calcula las próximas revisiones según el tiempo transcurrido o los kilómetros recorridos, ayudando al usuario a mantener el vehículo en óptimas condiciones.

3. Desarrollo del proyecto

3.1 Análisis del mercado y posible modelo de negocio

Como hemos comentado, hay aplicaciones similares pero suelen ser para flotas de camiones o empresas de alquiler. También hemos encontrado dos aplicaciones similares a este tipo de proyecto son Car Care y Simply Auto. Ambas están orientadas al registro y control del mantenimiento de vehículos, permitiendo al usuario guardar información sobre repostajes, revisiones, gastos y recordatorios de servicio. Ambas ofrecen historiales detallados del vehículo y sistemas de alertas para próximas revisiones, funcionando como herramientas de gestión personal del coche o moto.

Como ventajas, estas aplicaciones destacan por su gran cantidad de funcionalidades ya implementadas y por su madurez en el mercado, lo que las hace muy completas y estables. Como desventajas, suelen ser aplicaciones demasiado complejas para usuarios básicos, con interfaces recargadas y demasiadas opciones que no siempre son necesarias o intuitivas. Muchas de sus funciones más útiles están bloqueadas detrás de versiones de pago o suscripciones, lo que limita su accesibilidad en comparación con soluciones más simples y directas como la planteada en este proyecto, para el conductor normal, la oferta es escasa o poco amigable.

Nuestro modelo de negocio inicial para TranquiCar sería el modelo Freemium. La aplicación sería totalmente gratuita para gestionar un solo vehículo ideal para captar a la mayoría de los usuarios. Si el usuario quiere añadir más de un coche o una moto a su "Garaje", se le ofrecería un pequeño pago único o suscripción anual barata para desbloquear la versión Premium. Otra opción a futuro sería incluir pequeños banners de talleres mecánicos locales patrocinados.

3.2 Metodologías utilizadas

Aunque inicialmente el proyecto se planteó siguiendo la metodología Scrum, no fue posible aplicarla en su totalidad debido a cambios importantes en la composición del equipo durante el desarrollo. El proyecto comenzó con un grupo de seis integrantes, organizando las primeras tareas mediante Trello y definiendo los roles Scrum correspondientes. Sin embargo, a medida que avanzó el trabajo, cuatro integrantes abandonaron el proyecto, quedando únicamente dos personas responsables de completar todo el desarrollo.

Esta situación nos obligó a adaptar la metodología de trabajo de forma más flexible y menos estricta que en un entorno Scrum ideal. Aun así, mantuvimos algunos elementos fundamentales de la metodología, como la planificación de tareas, la división por sprints y el seguimiento del progreso mediante un tablero Kanban en Trello.

Debido a la reducción del equipo, asumimos una gran carga de trabajo y tuvimos que repartirnos las principales áreas del proyecto. Uno de los miembros se centró principalmente en el desarrollo backend y la lógica de la aplicación, mientras que el otro se encargó del frontend y la interfaz visual. A pesar de esta división, el trabajo se realizó de manera muy colaborativa, realizando pruebas conjuntas, revisando el trabajo del compañero y ayudándose mutuamente a resolver errores, dudas y problemas técnicos. Además, tuvimos que comprender y participar parcialmente en todas las partes del proyecto para poder mantener una visión global de la aplicación y asegurar una correcta integración entre frontend y backend. Esto permitió continuar el desarrollo de forma coordinada y completar el proyecto con éxito a pesar de las dificultades organizativas.

3.3 Descripción de los componentes de la aplicación

Arquitectura del fronted:

El frontend de la aplicación se ha desarrollado siguiendo una arquitectura modular y escalable, basada en React Native y gestionada mediante Expo. La estructura del proyecto se organiza en carpetas que separan las distintas responsabilidades como assets para los recursos gráficos, config para las configuraciones globales como la URL de la API, context para la gestión del estado global mediante el Context API, y screens para las diferentes pantallas de la aplicación. Esta organización permite mantener un código limpio, reutilizable y fácil de mantener, donde cada pantalla actúa como un componente independiente que se comunica con el backend a través de peticiones HTTP. La arquitectura sigue el patrón MVC adaptado a React, donde las vistas son los componentes, el controlador se representa en la lógica de cada pantalla y el modelo se gestiona desde el servidor.

Tecnologías utilizadas:

El desarrollo se ha realizado con React Native y Expo, aprovechando su capacidad para generar aplicaciones móviles multiplataforma como lo son Android e iOS con un único código base. Se han empleado React Hooks como useState, useEffect y useContext para la gestión de estados y ciclos de vida, y React Navigation para la navegación entre pantallas. La comunicación con el backend se realiza mediante la Fetch API, enviando y recibiendo datos en formato JSON desde rutas REST. Además, se integran librerías como Expo ImagePicker para la captura y selección de imágenes, y MaterialIcons e Ionicons para la iconografía. El uso de Alert API permite mostrar mensajes modales al usuario, garantizando una interacción clara y directa. Todo el diseño se implementa con StyleSheet, asegurando consistencia visual y mayor rendimiento.

Diseño y mockups:

El diseño de la interfaz de la aplicación sigue una línea minimalista y moderna, utilizando principalmente tonos oscuros combinados con acentos en rojo profundo, lo que aporta una estética profesional con un aire familiar automovilístico. La interfaz ha sido desarrollada

priorizando la claridad visual, la coherencia entre pantallas y la facilidad de uso para cualquier tipo de usuario. Las pantallas principales incluyen formularios de acceso y registro intuitivos y centrados, con campos amplios, botones destacados para mayor facilidad visual y presencia constante del logotipo de la aplicación para reforzar la identidad visual.

La pantalla principal muestra el “Garaje” del usuario mediante tarjetas de vehículos y un botón flotante para añadir nuevos elementos de forma rápida e intuitiva. Además, cada vehículo dispone de una vista detallada con información sobre mantenimientos y recordatorios próximos, utilizando tarjetas interactivas. Las pantallas de gestión de mantenimientos y recordatorios mantienen la misma coherencia visual y jerarquía tipográfica, facilitando una navegación fluida y organizada. Gracias a eso, la aplicación ofrece una experiencia de usuario clara e intuitiva, complementada con alertas y transiciones que proporcionan feedback inmediato durante la interacción.

Comunicación entre fronted y backend:

La comunicación entre el frontend y el backend de la aplicación se realiza mediante una API REST, utilizando rutas claramente definidas para gestionar las diferentes funcionalidades del sistema, como usuarios, vehículos, recordatorios y mantenimientos. Entre las principales rutas utilizadas se encuentran `/api/users`, `/api/vehicles`, `/api/reminders` y `/api/maintenance`, permitiendo una estructura organizada y escalable en el intercambio de información entre cliente y servidor.

El flujo de datos sigue una lógica sencilla y eficiente orientada a ofrecer una experiencia fluida al usuario. Los procesos de inicio de sesión y registro permiten autenticar o crear usuarios en el sistema, mientras que la pantalla principal carga automáticamente los vehículos asociados al usuario autenticado. Desde la vista individual de cada vehículo, se muestran los mantenimientos y recordatorios correspondientes, permitiendo añadir, editar o eliminar registros de forma dinámica mediante peticiones al servidor. Además, todas las interacciones importantes incluyen validaciones, alertas y navegación condicional, garantizando un funcionamiento seguro, intuitivo y coherente durante el uso de la

aplicación.

Arquitectura del backend:

El backend de la aplicación está construido siguiendo una arquitectura REST, organizada en módulos independientes que separan claramente las responsabilidades de cada parte del sistema. El servidor se basa en Node.js utilizando el framework Express, lo que permite una estructura ligera, flexible y fácilmente escalable.

La aplicación se divide en varios niveles, el servidor principal (Server.js), la configuración de Express (App.js), los modelos de datos definidos con Sequelize, las rutas que gestionan cada recurso como lo son, usuarios, vehículos, mantenimientos y recordatorios y la configuración de la base de datos. Esta organización modular facilita el mantenimiento, la lectura del código y la ampliación futura del proyecto.

Base de datos y ORM:

El backend de la aplicación utiliza PostgreSQL como sistema de gestión de bases de datos, proporcionando una estructura sólida y eficiente para el almacenamiento de la información. La conexión y gestión de la base de datos se realiza mediante el ORM Sequelize, lo que permite trabajar con modelos y relaciones de forma más sencilla y organizada, evitando la necesidad de escribir consultas SQL manuales en la mayoría de los casos. Esta tecnología facilita el mantenimiento del Código.

La aplicación se organiza principalmente a través de cuatro modelos fundamentales: User, encargado de representar a los usuarios registrados, Vehicle, que almacena los vehículos asociados a cada usuario, el cual es Maintenance, destinado al registro de mantenimientos realizados sobre cada vehículo y Reminder, utilizado para gestionar los recordatorios programados. Las relaciones entre estos modelos están correctamente definidas, permitiendo que un usuario tenga múltiples vehículos y que cada vehículo pueda contener varios mantenimientos y recordatorios. Esta estructura refleja un modelo de datos coherente, organizado y adaptado a las necesidades funcionales de la aplicación.

3.4 Principales problemas y soluciones

Dificultades relacionadas con el trabajo en equipo y la organización del desarrollo

Durante el desarrollo del proyecto, una de las principales dificultades estuvo relacionada con la reducción del tamaño del equipo. Aunque la idea inicial era desarrollar el proyecto siguiendo una metodología Scrum, el equipo terminó reduciéndose a la mitad. Esta situación obligó a replantear la organización del trabajo y adaptar la planificación inicial a una estructura mucho más reducida.

Debido a esta limitación, no fue posible aplicar Scrum de manera estricta, ya que esta metodología está pensada principalmente para equipos más amplios y con roles diferenciados, como Scrum Master, Product Owner y equipo de desarrollo. En este proyecto, ambos integrantes tuvieron que asumir múltiples responsabilidades simultáneamente, haciendo inviable mantener una estructura Scrum completa.

La organización del trabajo se adaptó de manera más flexible a las necesidades reales del proyecto. Las tareas se dividieron principalmente por áreas de especialización: uno de los integrantes se encargó principalmente del desarrollo frontend y del diseño de la aplicación móvil en React Native, mientras que otro se centró en el desarrollo backend, la API REST, la base de datos PostgreSQL y el despliegue del servidor.

Aunque esta división permitió avanzar de forma paralela sobre diferentes partes de la aplicación, también surgieron dificultades de coordinación entre frontend y backend. En varias ocasiones fue necesario modificar endpoints, adaptar estructuras de datos o ajustar la lógica de funcionamiento para garantizar la correcta comunicación entre ambas partes de la aplicación.

Otro problema frecuente fue la dependencia entre tareas. Algunas funcionalidades del frontend no podían completarse hasta que determinadas rutas o modelos backend estuvieran terminados, obligando a reorganizar prioridades y realizar ajustes constantes durante el desarrollo.

A pesar de no aplicar Scrum de manera formal, sí se mantuvo una organización inspirada en metodologías ágiles, basada en la comunicación constante entre los integrantes, la división de objetivos parciales y la revisión periódica del estado del proyecto. Esta forma de trabajo permitió detectar errores rápidamente, solucionar problemas técnicos de forma

progresiva y adaptar funcionalidades según las necesidades que iban apareciendo durante el desarrollo.

Gracias a esta organización flexible y a la colaboración continua entre frontend y backend, fue posible completar un producto funcional y estable, manteniendo una buena coordinación entre ambas partes del sistema y resolviendo satisfactoriamente las dificultades técnicas encontradas durante el proyecto.

Durante el desarrollo del proyecto surgieron diferentes dificultades tanto a nivel frontend como backend. La mayoría de estos problemas estuvieron relacionados con la comunicación entre la aplicación móvil y la API REST, la gestión de estados en React Native y la sincronización de la lógica de mantenimientos y recordatorios.

Problemas relacionados con la comunicación entre frontend y backend:

Uno de los primeros problemas detectados fue la dificultad para conectar correctamente la aplicación móvil desarrollada en React Native con el backend en Node.js y Express.

Inicialmente, las peticiones HTTP se realizaban con direcciones IP locales, lo que provocaba errores de conexión cuando se cambiaba de red o se intentaba acceder desde otros dispositivos.

Para solucionar este problema, se desplegó el backend en la plataforma cloud Render y se centralizó la URL de la API en un archivo de configuración. Esto permitió realizar todas las peticiones mediante una dirección pública accesible desde cualquier red.

También aparecieron errores relacionados con peticiones HTTP mal construidas, especialmente durante la migración desde direcciones locales a la URL pública del backend. Algunos errores se debían al uso incorrecto de template literals en JavaScript, provocando fallos de tipo "Network request failed". Estos problemas se solucionaron revisando y corrigiendo todas las llamadas fetch del proyecto.

Problemas con React Native y la gestión de componentes:

Durante el desarrollo de la pantalla de recordatorios surgió un problema relacionado con el teclado en dispositivos móviles. Cada vez que el usuario escribía un número en un campo

de texto, el teclado se cerraba automáticamente, obligando a escribir carácter por carácter.

La causa de este problema era la definición de componentes internos dentro del propio componente principal, provocando que React Native desmontara y recreara los inputs en cada renderizado. La solución consistió en reorganizar la estructura del código y mover dichos componentes fuera del componente principal para evitar renderizados innecesarios. Otro problema importante estuvo relacionado con la navegación entre pantallas y el paso de parámetros mediante React Navigation. En algunas ocasiones los datos seleccionados no llegaban correctamente a otras pantallas, especialmente en el flujo entre recordatorios y mantenimientos. Para solucionarlo, se revisó la navegación y el uso de `props.route.params`, asegurando que los objetos seleccionados se enviaran correctamente entre pantallas.

Problemas relacionados con Sequelize y PostgreSQL:

Durante la implementación de la base de datos surgieron errores relacionados con asociaciones entre modelos y sincronización de tablas en Sequelize. Algunos problemas aparecieron debido a diferencias entre nombres de columnas, relaciones entre entidades o configuraciones incorrectas de claves foráneas.

Estos problemas se solucionaron revisando la estructura de los modelos y asegurando la correcta relación entre usuarios, vehículos, mantenimientos y recordatorios mediante claves externas y asociaciones Sequelize.

Problemas no resueltos completamente, de lógica y sincronización de datos:

Uno de los mayores retos del proyecto fue diseñar correctamente la relación entre mantenimientos realizados y kilometraje. Los recordatorios y los mantenimientos siguen funcionando como elementos más o menos independientes, generando inconsistencias en los cálculos de próximos mantenimientos.

Además hemos sido incapaces de encontrar una solución efectiva al seguimiento del kilometraje.

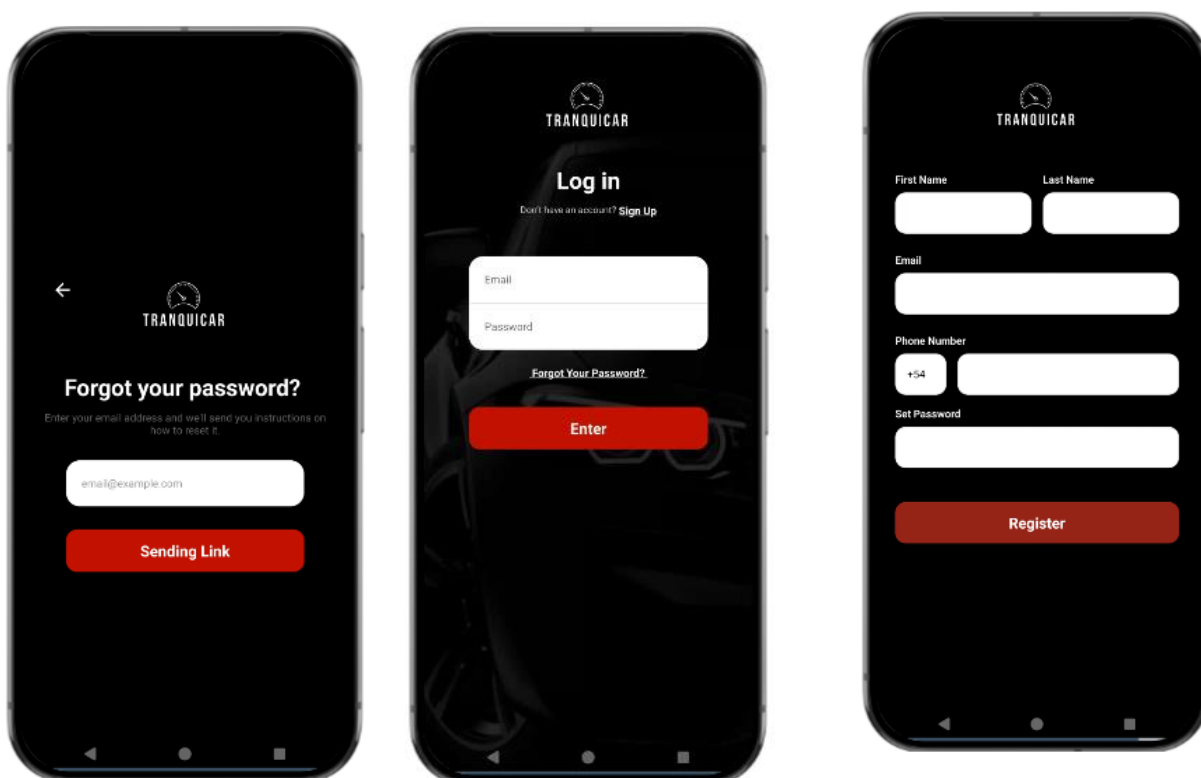
3.5 Resultados obtenidos

Enlace Figma:

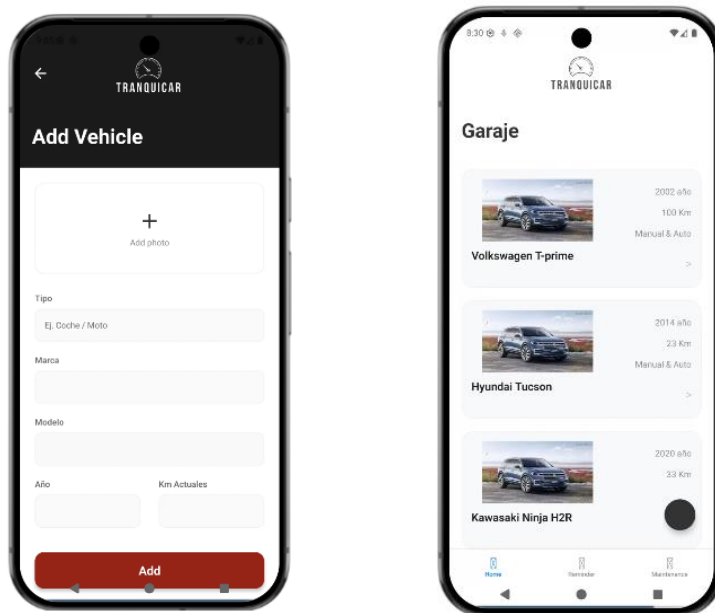
https://www.figma.com/design/ND6rhTwCTQkEruDihuuvtN/PI_DAM2?node-id=0-1&t=12UwZuZ1hlzi4coa-1

Estamos muy orgullosos del resultado final. Aquí adjuntamos cómo han quedado las principales pantallas funcionales de la aplicación:

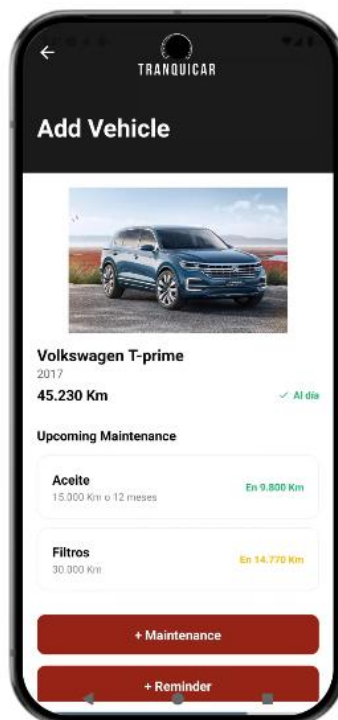
- **Pantallas de Inicio, Registro y Recuperar contraseña:** La app cuenta con una pantalla de carga tipo Splash súper elegante y unos formularios limpios para registrarse ("Sign in" / "Register"/"Forgot your Password") pidiendo solo los datos necesarios.



- **El Garaje (Home):** Es la pantalla central donde listamos los vehículos mediante tarjetas. Como se ve, te marca de forma visual el modelo, los kilómetros totales y un indicador rápido y colorido.

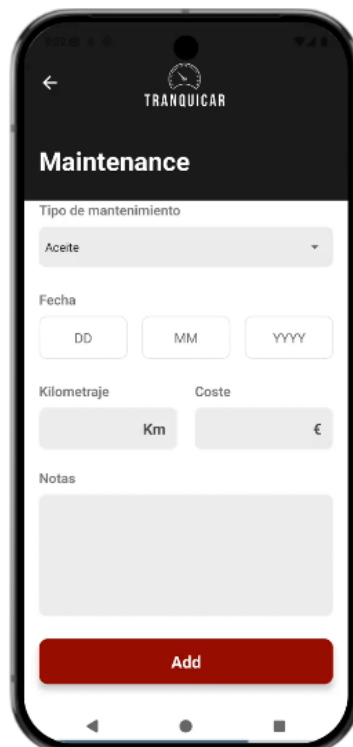
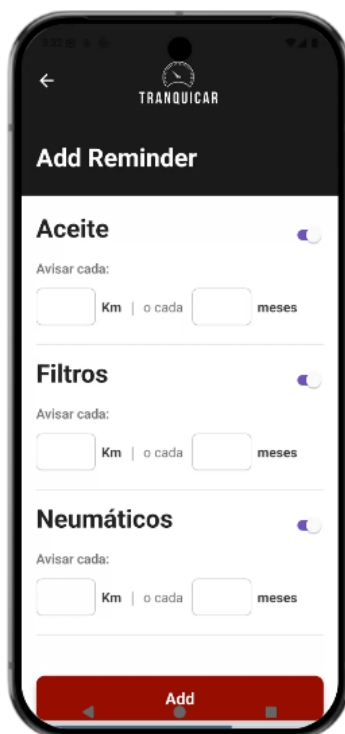
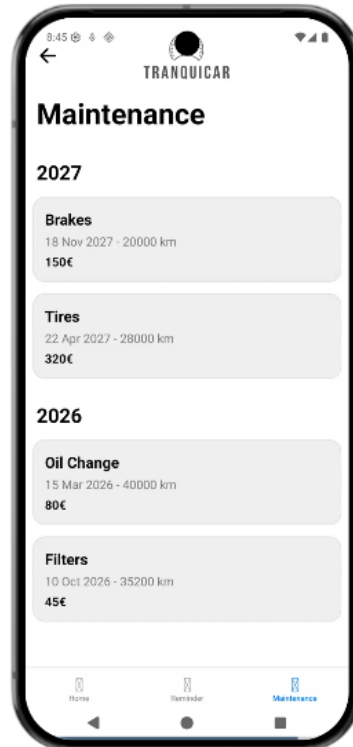
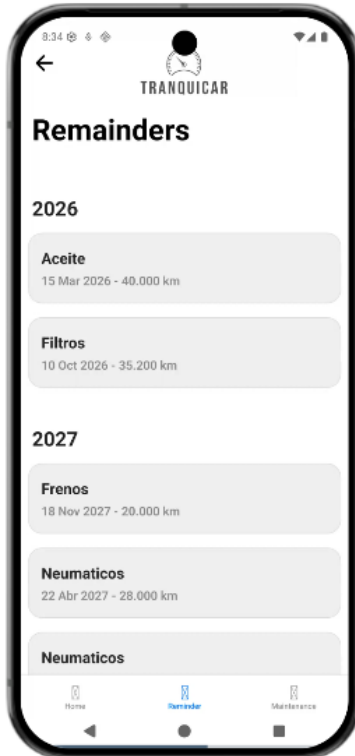


- **Detalle del Vehículo y Mantenimiento:** Al entrar en un vehículo, vemos su Dashboard específico con los mantenimientos y recordatorios próximos y sus detalles como la marca, kilómetros y año, alertándote por colores si estas al día o necesita revisión.



- **Gestión y Recordatorios:** Tenemos los formularios para añadir recordatorios y/o mantenimientos. La pantalla de "Reminders" permite mostrar los recordatorios

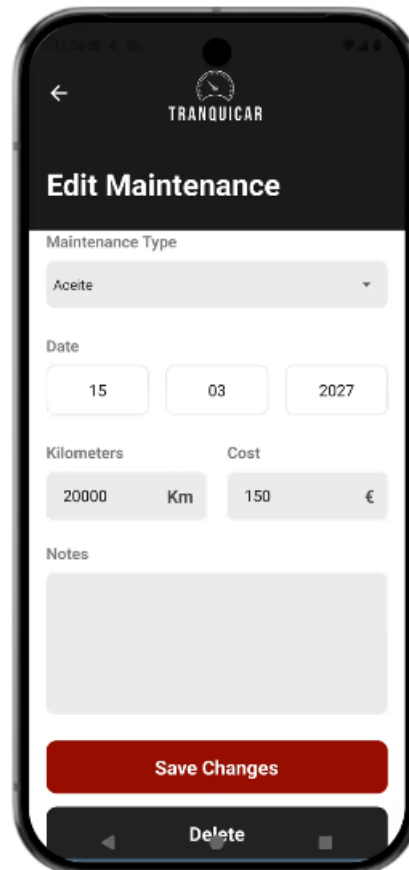
configurados como alarmas personalizadas activando interruptores y definiendo cada cuántos Km o meses queremos que la app nos avise, teniendo un historial de los recordatorios por vehículo al igual con la pantalla “Maintenance”.



- **Gestión y Recordatorios:** Tenemos los formularios para añadir recordatorios y/o mantenimientos. La pantalla de "Reminders" permite mostrar los recordatorios configurados como alarmas personalizadas activando interruptores y definiendo cada cuántos Km o meses queremos que la app nos avise, teniendo un historial de los recordatorios por vehículo al igual con la pantalla "Maintenance".



The 'Edit Reminder' screen features a dark header with a back arrow and the 'TRANQUICAR' logo. The title 'Edit Reminder' is centered. Below, there are three input fields: 'Título' with the value 'Aceite', 'Fecha' with '15 Mar 2026', and 'Kilómetros' with '40.000 km'. At the bottom, there are two buttons: a red 'Guardar cambios' button and a grey 'Eliminar reminder' button.



The 'Edit Maintenance' screen has a dark header with a back arrow and the 'TRANQUICAR' logo. The title 'Edit Maintenance' is centered. The form includes a 'Maintenance Type' dropdown menu set to 'Aceite', a 'Date' section with three input boxes for '15', '03', and '2027', and a 'Kilometers' section with '20000' and 'Km'. The 'Cost' section has '150' and '€'. Below these is a 'Notes' text area. At the bottom, there are two buttons: a red 'Save Changes' button and a grey 'Delete' button.

4. Conclusiones

A nivel técnico, el proyecto ha supuesto una experiencia muy completa en el desarrollo full stack. Se han aplicado conocimientos relacionados con frontend móvil mediante React Native y Expo, backend con Node.js y Express, bases de datos relacionales con PostgreSQL y gestión de modelos mediante Sequelize. Además, también se ha trabajado el despliegue cloud del backend utilizando Render, permitiendo convertir una idea en una aplicación.

Desde un punto de vista académico, el proyecto ha servido para consolidar y aplicar conocimientos adquiridos durante los cursos, enfrentándose a situaciones reales de desarrollo, depuración y despliegue de software. Además, ha permitido comprender mejor el ciclo completo de desarrollo de una aplicación, desde la planificación inicial.

En el ámbito profesional y personal, ha permitido desarrollar capacidades de trabajo autónomo, aprendizaje continuo y adaptación frente a dificultades técnicas surgidas durante el desarrollo.

5. Líneas de trabajo futuro

La aplicación actual se trata de un MVP (producto mínimo viable) existen muchísimas mejoras y ampliaciones futuras, la aplicación actual permite validar la utilidad de la idea y ofrecer una experiencia mínimamente funcional al usuario final.

Una de las mejoras más importantes sería la implementación de un sistema de autenticación más avanzado mediante JWT (JSON Web Tokens). Actualmente, el sistema de login permite acceder a la aplicación de forma funcional, pero la incorporación de tokens de autenticación permitiría aumentar la seguridad, mantener sesiones persistentes y proteger mejor las rutas privadas del backend. Para implementar esta mejora sería necesario añadir middleware de autenticación en Express y almacenamiento seguro de tokens en el dispositivo móvil.

Otra línea de trabajo futura consistiría en añadir notificaciones push automáticas.

Actualmente, los recordatorios y alertas solo se visualizan dentro de la aplicación. En futuras versiones, el sistema podría enviar avisos automáticos al móvil cuando un mantenimiento esté próximo por fecha o kilometraje. Para ello sería necesario integrar servicios como Expo Notifications, además de implementar lógica de comprobación automática de recordatorios desde el backend.

También sería interesante desarrollar un sistema de estadísticas y análisis de gastos, permitiendo al usuario visualizar gráficos relacionados con el coste de mantenimiento de cada vehículo, frecuencia de revisiones o evolución del kilometraje. Esta funcionalidad requeriría incorporar librerías de visualización gráfica y ampliar la lógica de consultas sobre la base de datos.

A nivel técnico, una futura evolución del proyecto podría incluir una arquitectura backend más escalable, separando la lógica en: controladores, servicios y middleware independientes. Esto permitiría mejorar la mantenibilidad y organización del código a medida que aumente el tamaño del proyecto.

El objetivo es transformar la aplicación desde un MVP funcional hacia una plataforma de gestión de mantenimiento de vehículos mucho más completa, escalable y orientada a usuarios reales.

6. Bibliografia

React Native. (n.d.). Introduction · React Native. React Native Documentation.

<https://reactnative.dev/docs/getting-started>

Expo. (n.d.). Introduction to Expo. Expo Documentation. <https://docs.expo.dev/>

React Navigation. (n.d.). Getting Started. React Navigation Documentation

<https://reactnavigation.org/docs/getting-started>

Expo. (n.d.). Icons. Expo Documentation. <https://docs.expo.dev/guides/icons/>

Node.js. (n.d.). Node.js Documentation. <https://nodejs.org/docs/latest/api/>

Express.js. (n.d.). Routing. Express.js Documentation.

<https://expressjs.com/en/guide/routing.html>

Sequelize. (n.d.). Sequelize v6 Documentation <https://sequelize.org/docs/v6/>

PostgreSQL Global Development Group. (n.d.). PostgreSQL Documentation.

<https://www.postgresql.org/docs/current/index.html>

MDN Web Docs. (n.d.). Using Fetch API. [https://developer.mozilla.org/en-](https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch)

[US/docs/Web/API/Fetch_API/Using_Fetch](https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch)

Render. (n.d.). Render Web Services Documentation. <https://render.com/docs/web-services>

Render. (n.d.). Render PostgreSQL Documentation. <https://render.com/docs/postgresql>

Expo. (n.d.). ImagePicker. Expo Documentation.

<https://docs.expo.dev/versions/latest/sdk/imagepicker/>

Expo. (n.d.). Notifications. Expo Documentation

<https://docs.expo.dev/versions/latest/sdk/notifications/>

npm. (n.d.). jsonwebtoken. <https://www.npmjs.com/package/jsonwebtoken>

Meta Platforms, Inc. (n.d.). React Hooks API Reference. React Documentation.

<https://react.dev/reference/react>

Trello. (n.d.). Trello Guide. <https://trello.com/guide>

JSON.org. (n.d.). Introducing JSON. <https://www.json.org/json-en.html>

REST API Tutorial. (n.d.). RESTful API Tutorial. <https://restfulapi.net/>

GitHub. (n.d.). GitHub Documentation. <https://docs.github.com/>

Mozilla Developer Network. (n.d.). JavaScript Template literals.

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Template_literals

React Native. (n.d.). Alert API. React Native Documentation.

<https://reactnative.dev/docs/alert>

React Native. (n.d.). StyleSheet API. React Native Documentation.

<https://reactnative.dev/docs/stylesheet>